Babel-Player — Engineering Milestone Tracker

Consolidated Implementation Plan: Codebase Health · Inference Performance · NeMo Provider Integrations

Category: Engineering Report | Author: Michael | Date: April 7, 2026 | Status: Active

---

Project Context

Babel-Player is a local-first multilingual video dubbing application. Stack: C# / .NET 10 / Avalonia 12.0.0 / CommunityToolkit.Mvvm. The pipeline runs three inference stages — Transcription, Translation, and TTS — with provider registries per stage supporting CPU, GPU, and Cloud compute profiles. All local inference runs through containerized Docker services or native subprocesses, with zero data leaving the machine on local profiles.

Phase 1: Foundation Stabilization

Low-risk fixes that eliminate silent bugs and reduce risk before larger changes. Estimated effort: 3–5 days.

1.0 — Upgrade to Avalonia 12.0.0 Stable

DO FIRST

— Establishes a stable API surface before all other work.

- **Problem:** The project targets Avalonia 12.0.0-rc1. Avalonia 12.0.0 stable was released on April 7, 2026 with fixes since rc1: CompositionAnimation fixes, accessibility backend fixes (Linux AT-SPI, macOS), TextPresenter measurement with trailing whitespace, WindowState now a direct property with reliable values, and focus/access key fixes. The AvaloniaUI.DiagnosticsSupport hang noted in the codebase may be resolved.
- **Fix:** Bump all Avalonia package references from 12.0.0-rc1 to 12.0.0 in `BabelPlayer.csproj`. Re-enable AvaloniaUI.DiagnosticsSupport and test. Review the breaking changes doc (WindowState is now a direct property — check any code that observes WindowState changes).
- **Risk:** Low. rc1 to stable is a patch-level change. The breaking changes between rc1 and rc2 (WindowState direct property) should be verified.
- **Files:** `BabelPlayer.csproj`, any code referencing WindowState

1.1 — Fix _mediaSnapshotCache Thread Safety

- **Problem:** `_mediaSnapshotCache` is a plain `Dictionary<string, WorkflowSessionSnapshot>` accessed from multiple threads (UI thread via Initialize/LoadMedia, background Task.Run via ApplyBootstrapWarmupData, CacheMediaSnapshot). Concurrent modification will corrupt state or throw.
- **Fix:** Replace with `ConcurrentDictionary<string, WorkflowSessionSnapshot>`. The project already uses ConcurrentDictionary for TTS segment paths, so the pattern is established.
- **Risk:** None. Drop-in replacement.
- **Files:** `SessionWorkflowCoordinator.cs`

1.2 — Create Fire-and-Forget Async Helper

- **Problem:** Multiple `_ = SomeAsyncMethod()` call sites (RefreshSegmentsAsync, SeekAndPlayAsync, PlayTtsForSegmentAsync, RefreshProviderReadinessStatusesAsync). These rely on the global OnUnobservedTaskException handler, which marks them observed and

shows a crash dialog — but the call site has zero awareness of failure. No status update, no retry, no user-facing message.

- **Fix:** Create a `FireAndForget(Task task, string context)` helper that logs exceptions via AppLog and optionally updates StatusText. Replace all `_ =` fire-and-forget patterns with this helper.
- **Risk:** Low. Improves observability of all subsequent work.
- **Files:** New utility class + all call sites in `EmbeddedPlaybackViewModel.cs`, `SessionWorkflowCoordinator.*.cs`

1.3 — Extract Duplicated Startup Code

- **Problem:** The coordinator construction block in `App.axaml.cs` (~40 lines) is copy-pasted into the catch handler. Both paths construct the full coordinator with all dependencies. If a probe, registry, or manager constructor changes, both paths must be updated.
- **Fix:** Extract a `CreateCoordinator()` factory method. Call it in the try block; call it again in catch with a flag to skip the component that failed.
- **Risk:** None.
- **Files:** `App.axaml.cs`

1.4 — Fix Hardcoded Language Fallback

- **Problem:** In `RegenerateSegmentTranslationAsync`: `var sourceLanguage = CurrentSession.SourceLanguage ?? "es";` — Spanish as a default source language is arbitrary. If a session has no source language recorded, this silently picks Spanish.
- **Fix:** Throw or block with a clear message if SourceLanguage is null at this point. The transcription step should always set it. A null here indicates a state machine violation.
- **Risk:** None. Converts a silent bug into an explicit error.
- **Files:** `SessionWorkflowCoordinator.Pipeline.cs`

1.5 — Stream TTS Audio Downloads

- **Problem:** `DownloadTtsAudioAsync` in `ContainerizedInferenceClient` buffers entire response in memory via `ReadAsByteArrayAsync` before writing to disk. For large segments or combined audio, this creates unnecessary memory pressure.
- **Fix:** Replace with `ReadAsStreamAsync` piped to `File.Create` via `CopyToAsync`. Two-line change.
- **Risk:** None.
- **Files:** `ContainerizedInferenceClient.cs`

```
await using var responseStream = await response.Content.ReadAsStreamAsync(cancellationToken); await using var fileStream = File.Create(localOutputPath); await responseStream.CopyToAsync(fileStream, cancellationToken);
```

1.6 — Reuse HttpClient for Cloud Providers

- **Problem:** `OpenAiTtsProvider` and `ElevenLabsTtsProvider` call `using var client = _clientFactory();` inside every `GenerateSegmentTtsAsync` call. The `using` disposes the client after each segment, paying TCP connection setup + TLS handshake per segment.
- **Fix:** Create the API client once in the provider constructor (or lazily on first use) and reuse it across all segment calls within a session. HttpClient is designed to be long-lived.
- **Risk:** Very low. Standard HttpClient usage pattern.
- **Files:** `OpenAiTtsProvider.cs`, `ElevenLabsTtsProvider.cs`

Phase 2: TTS Performance Quick Wins

Direct inference time reductions with minimal architectural change. Estimated effort: 3–5 days.

2.1 — Eliminate Double TTS Synthesis

HIGHEST PRIORITY — Single biggest performance win available.

- **Problem:** `GenerateTtsAsync` in `SessionWorkflowCoordinator.Pipeline.cs` synthesizes every segment twice. Pass 1 calls `_ttsService.GenerateTtsAsync()` to produce the combined audio file — for XTTS and Qwen this iterates through every segment in a sequential foreach, synthesizes each one, downloads it, then ffmpeg-concatenates results. Pass 2 calls `Parallel.ForEachAsync` over the same segments to produce per-segment clips. That is 2x the inference cost.
- **Fix:** Flip the order. Generate per-segment clips first (already parallelized), then produce the combined audio by ffmpeg-concatenating the segment files. The combined pass becomes a pure I/O operation — zero additional inference. Extract the concat logic already present in XttsContainerTtsProvider and QwenContainerTtsProvider to a shared utility and call it from the coordinator after the parallel segment pass.
- **Impact:** ~50% TTS wall-clock time reduction. Eliminates the sequential foreach bottleneck in XTTS/Qwen GenerateTtsAsync entirely.
- **Files:** `SessionWorkflowCoordinator.Pipeline.cs`, `XttsContainerTtsProvider.cs`, `QwenContainerTtsProvider.cs`

2.2 — Provider-Aware Parallelism Cap

- **Problem:** Per-segment `Parallel.ForEachAsync` is capped at `Math.Max(1, Math.Min(4, Environment.ProcessorCount / 2))`. Reasonable for local single-GPU inference, but leaves throughput on the table for cloud providers (network-bound, not compute-bound — could safely run 8–16 concurrent requests) and some containerized providers.
- **Fix:** Add a `MaxConcurrency` property (with a default implementation) to `ITtsProvider`. Cloud providers override to return higher values (e.g., 12). The coordinator uses this instead of the hardcoded cap.
- **Impact:** ~30% TTS speedup for cloud providers (OpenAI, ElevenLabs). No change for local providers.
- **Files:** `ITtsProvider.cs`, `OpenAiTtsProvider.cs`, `ElevenLabsTtsProvider.cs`, `SessionWorkflowCoordinator.Pipeline.cs`

```
int parallelism = Math.Max(1, Math.Min(     _ttsService.MaxConcurrency,
candidateSegments.Count));
```

Phase 3: Diarization Provider Overhaul

Eliminate HuggingFace token friction with NeMo MSDD (primary) and WeSpeaker (fallback). Estimated effort: 1.5–2 weeks.

3.1 — Replace Reflection-Based Diarization Discovery

- **Problem:** `GetRegisteredDiarizationProviderNames()` probes `DiarizationRegistry` via System.Reflection — looking for ProviderNames, AvailableProviderNames, or Providers properties. This is brittle, not compile-time safe, and breaks the pattern of explicit registration used by transcription, translation, and TTS registries.

- **Fix:** Give `IDiarizationRegistry` the same `GetAvailableProviders()` contract the other registries have and inject it like everything else. This is a prerequisite for cleanly adding NeMo and WeSpeaker providers.
- **Risk:** Low. Aligns diarization with the established pattern.
- **Files:** `IDiarizationRegistry` (new or updated interface), `DiarizationRegistry`, `SessionWorkflowCoordinator.cs`

3.2 — Add WeSpeaker Diarization Provider

- **Purpose:** Validates the new multi-provider diarization pattern end-to-end with minimal risk. Also serves as the permanent lightweight CPU fallback.
- **Implementation:**
  - Build a slim Docker container (~2–3GB): python:3.10-slim + wespeaker + torch + torchaudio. Pre-download the english model at build time.
  - FastAPI wrapper: single POST /diarize endpoint accepting audio file upload, returning JSON segments [{start, end, speaker}, ...].
  - C# WeSpeakerDiarizationProvider implementing IDiarizationProvider, following the same containerized HTTP pattern as other providers.
- **Quality:** Clustering-only (no neural refinement), DER ~12–16%. Adequate for content with clear speaker turns. Lower accuracy on overlapping speech.
- **License:** Apache 2.0. Fully AGPL-compatible.
- **Files:** New Docker container, new provider class, registry updates

3.3 — Build NeMo MSDD Slim Docker Container

- **Purpose:** Primary diarization provider. DER ~8.1% avg vs pyannote's ~18%.
- **Implementation:**
  - Base image: pytorch/pytorch:2.4.1-cuda12.1-cudnn9-runtime
  - Install only nemo_toolkit[asr], hydra-core, omegaconf, wget

    - Pre-download MarbleNet (VAD) + TitaNet-Large (embeddings) + MSDD models from NGC at build time — no authentication required, all Apache 2.0
    - FastAPI wrapper: POST /diarize endpoint, same contract as WeSpeaker for provider interface compatibility
- **Target image size:** ~6–8GB (comparable to existing XTTS container).
- **Key advantage:** Models are baked into the container at build time. Inference is fully air-gapped — no internet required at runtime.
- **License:** Apache 2.0 throughout. No gated models, no tokens.

3.4 — Add NemoDiarizationProvider (C#)

- **Implementation:** Same containerized HTTP pattern as other providers. Sends WAV audio via multipart POST to the NeMo container's /diarize endpoint. Parses JSON segments. Maps to existing DiarizationResult model.
- Wire into InferenceRuntimeCatalog with `ComputeProfile.ContainerGpu`.
- RTTM output maps directly to the segment format used downstream for per-speaker audio isolation (XTTS voice cloning).
- **Files:** New provider class, InferenceRuntimeCatalog updates

3.5 — Diarization Provider Selection UI

- **Auto-detection logic:** GPU detected → default to NeMo MSDD. CPU-only → default to WeSpeaker. User can override in settings.
- Surface both providers in the existing provider dropdown pattern used by transcription/translation/TTS.
- **Estimated effort:** ~1 day (UI already has the pattern).

3.6 — pyannote Legacy Decision

- **Decision point:** Keep pyannote-local as a third option for users who already have HuggingFace tokens, or deprecate it.

- **Recommendation:** Keep it for one release cycle as a fallback, then deprecate. NeMo MSDD is strictly better on accuracy, licensing, and friction.

Phase 4: ASR Provider Expansion (NeMo Parakeet)

Add Parakeet-TDT-0.6B-v3 as a high-performance ASR option for European languages. Estimated effort: 2–3 weeks.

4.1 — Build Slim Parakeet Container

- **Base image:** `pytorch/pytorch:2.4.1-cuda12.1-cudnn9-runtime`
- Install `nemo_toolkit[asr]` and pre-download `parakeet-tdt-0.6b-v3` from NGC at build time (no auth required).
- FastAPI wrapper: `POST /transcribe` endpoint accepting audio file upload, returning JSON segments with timestamps.
- **Target image size:** ~6–8GB.
- **License:** CC-BY-4.0. Compatible with AGPL-3.0.

4.2 — Add ParakeetTranscriptionProvider (C#)

- Implement `ITranscriptionProvider` following the containerized HTTP pattern.
- Wire into InferenceRuntimeCatalog with `ComputeProfile.ContainerGpu`.
- **Key advantage:** Roughly 10x faster than faster-whisper with lower WER (6.05% vs 7.44% on standard benchmarks).
- **Files:** New provider class, InferenceRuntimeCatalog updates

4.3 — Language Routing Logic

- **Problem:** Parakeet supports 25 European languages. faster-whisper supports 100+. Parakeet should be used when it can, with faster-whisper as the universal fallback.

- **Option A (simpler):** Expose both providers in the UI dropdown. Let the user pick based on their content language.
- **Option B (smarter):** Auto-detect source language from a short audio sample (first 30 seconds). If detected language is in Parakeet's supported set → route to Parakeet. Otherwise → faster-whisper.
- **Recommendation:** Ship Option A first. Add Option B as a follow-up once both providers are stable.

4.4 — Integration Testing

- **Test matrix:** Verify transcription quality and segment format compatibility across the language boundary (Parakeet's 25 EU languages vs faster-whisper's full set).
- Confirm downstream pipeline stages (translation, TTS) work identically regardless of which ASR provider produced the segments.
- **Regression test:** Ensure the session snapshot model (`SessionWorkflowStage.Transcribed`) works correctly with both providers.

Phase 5: Medium-Term Pipeline Optimizations

Process lifecycle and batching improvements. Estimated effort: 1–2 weeks.

5.1 — Batch Python Scripts for Edge TTS and Piper

- **Problem:** EdgeTtsProvider and PiperTtsProvider each spawn a new Python process per segment. For a 200-segment video, that is 200 Python interpreter startups with ~0.5–1.5 seconds of overhead per segment from process creation and import statements.
- **Fix (Edge TTS):** Write a batch-mode Python script that reads a JSON array of `{text, outputPath, voice}` items from stdin and processes them all in one interpreter session.

- **Fix (Piper):** Piper's CLI supports streaming from stdin line-by-line. Spawn once, pipe segments sequentially. Split output by segment boundaries.
- **Impact:** ~20% TTS speedup for Edge TTS and Piper providers.
- **Files:** New Python batch scripts, `EdgeTtsProvider.cs`, `PiperTtsProvider.cs`

5.2 — Persistent Python Worker Pool

- **Problem:** Even with batching, the subprocess-per-session model still pays Python startup cost per session.
- **Fix:** Replace `PythonSubprocessServiceBase` with a `PythonWorkerPool` — a pool of long-lived Python worker processes that accept work over stdin/stdout JSON-RPC. Workers stay warm, imports paid once, pool size controls concurrency.
- **Impact:** Eliminates ~1s Python startup + ~0.3s import overhead per invocation. Eliminates temp script file I/O.
- **Prerequisite:** Phase 5.1 (batch scripts validate the multi-segment-per-process pattern first).
- **Files:** `PythonSubprocessServiceBase.cs` (replaced), new `PythonWorkerPool` class

Phase 6: Long-Term Architectural Refactors

Major structural changes that benefit from all prior stabilization. Estimated effort: 4–8 weeks total, can be staggered.

6.1 — Decompose EmbeddedPlaybackViewModel

- **Problem:** The ViewModel handles playback controls, pipeline execution, provider/model/voice selection for all three stages, multi-speaker routing, dub mode, subtitle management, provider readiness polling, and segment

inspection. 1200+ lines with ~50 observable properties. Every new feature touches this file.

- **Fix:** Extract focused sub-ViewModels: PipelineConfigViewModel, DubModeViewModel, MultiSpeakerViewModel, SubtitleViewModel. Compose them in EmbeddedPlaybackViewModel. The coordinator already has this decomposition pattern (seven partial classes by concern) — the ViewModel layer should mirror it.
- **Impact:** Reduces merge conflicts, improves testability, makes the ViewModel layer match the coordinator's clean separation.
- **Files:** `EmbeddedPlaybackViewModel.cs` (split), new sub-ViewModel classes

6.2 — Streaming Pipeline with Inter-Stage Overlap

- **Problem:** `AdvancePipelineAsync` is strictly sequential: Transcribe (all segments) → Translate (all segments) → TTS (all segments). Each stage waits for the previous one to fully complete.
- **Fix:** Use `System.Threading.Channels.Channel<T>` as segment queues between stages. Each stage consumes from its input channel and produces to its output channel, creating a streaming producer-consumer chain.
- **Prerequisite:** Containerized transcription endpoint would need to support streaming responses (SSE or chunked JSON) to get full benefit. Translation via CTranslate2/NLLB can process segments individually — maps cleanly.
- **Consideration:** The session snapshot model assumes stage-level completion (`SessionWorkflowStage.Transcribed`, `.Translated`, `.TtsGenerated`). A streaming pipeline needs a more granular progress model.
- **Impact:** For a 10-minute video with ~200 segments, total pipeline time reduction of 30–50% by overlapping the long tails of each stage.
- **Files:** `SessionWorkflowCoordinator.Pipeline.cs`, all provider interfaces, session state model

6.3 — Server-Side Batching for XTTS/Qwen

- **Problem:** Each segment sends an individual HTTP request to the containerized Python server. The server processes one request at a time. GPU is idle between requests waiting for HTTP overhead and file I/O.
- **Fix:** Add a batch endpoint (`POST /tts/xtts/batch`) that accepts an array of segments and returns audio paths. The server batches text inputs into a single forward pass (XTTS v2 supports batched inference with padding). On the C# side, add a `GenerateBatchSegmentTtsAsync` method to ITtsProvider, group segments into batches of 4–8 (tunable by VRAM).
- **Impact:** GPU utilization from ~40% to 80%+. Biggest win for XTTS/Qwen on GPU.
- **Files:** Python inference server, `XttsContainerTtsProvider.cs`, `QwenContainerTtsProvider.cs`, `ITtsProvider.cs`

6.4 — Constructor Overload Cleanup

- **Problem:** SessionWorkflowCoordinator has two constructors — a convenience wrapper delegating to a primary that takes 14+ parameters. Readability and testability degrade as service count grows.
- **Fix:** Introduce a builder/factory pattern or a lightweight DI container (even Microsoft.Extensions.DependencyInjection). Replace the convenience overload with named arguments or an options record.
- **Files:** `SessionWorkflowCoordinator.cs`, `App.axaml.cs`

6.5 — Clean Shutdown (Replace Environment.Exit)

- **Problem:** `Environment.Exit(e.ApplicationExitCode)` is used because background threads (mpv event loop, debounce continuations, bootstrap warmup) keep the CLR alive. This is a symptom — Dispose() doesn't fully clean up all managed threads.
- **Fix:** Track all long-lived background operations with CancellationTokenSource instances owned by the coordinator. Cancel them in Dispose(). The mpv event loop likely needs a Quit command before disposal.

- **Files:** `App.axaml.cs`, `SessionWorkflowCoordinator.cs`, mpv integration layer

Watch List — No Action, Monitor for Changes

Technologies to revisit when licensing or access conditions change.

Canary-1B-v2 (ASR + Translation in One Pass)

- **What:** NVIDIA's Canary model performs ASR and translation in a single forward pass, collapsing two pipeline stages into one. Architecturally the biggest possible pipeline optimization.
- **Blocker:** CC-BY-NC-4.0 license — non-commercial only. Incompatible with AGPL distribution to commercial users.
- **Watch for:** NVIDIA relicensed Parakeet from restrictive (v2) to CC-BY-4.0 (v3). If Canary follows the same path, immediately prioritize integration.

Magpie TTS (Zero-Shot Voice Cloning)

- **What:** NVIDIA's Magpie TTS family includes zero-shot voice cloning models (Zeroshot, Flow).
- **Blockers:** English-only for cloning (+ European Spanish on Zeroshot). Gated access — "Apply for Access" on NIM, 2–3 day HuggingFace approval with non-commercial checkbox. Trades XTTS's 17-language zero-shot cloning for a narrower, gated, non-cloning alternative.
- **Watch for:** Multilingual voice cloning support + ungated permissive licensing.

DiariZen

- **What:** Open-source diarization with 30–50% DER reduction vs pyannote across benchmarks.

- **Blockers:** (1) Uses a pyannote-audio git submodule fork whose token bypass status is unverified — the fork's README still references HuggingFace access tokens. (2) Best model (wavlm-large-s80-md) uses CC-BY-NC-4.0 weights. The MIT-licensed base model has notably lower accuracy.
- **Watch for:** Confirmation that the fork eliminates the token requirement + a permissive license on the large model.

Appendix A: Priority Impact Matrix

Optimization	TTS Speedup	Pipeline Speedup	Effort	Risk	Phase
Fix thread safety	—	—	Trivial	None	1
Fire-and-forget helper	—	—	Low	Low	1
Extract startup code	—	—	Low	None	1
Fix "es" fallback	—	—	Trivial	None	1
Stream downloads	~5%	~2%	Trivial	None	1
Reuse HttpClient	~10% (cloud)	~4%	Trivial	None	1
Eliminate double synthesis	**~50%**	**~20%**	Low	Low	2
Provider-aware parallelism	~30% (cloud)	~10%	Very low	Low	2
Diarization registry fix	—	—	Low	Low	3
WeSpeaker provider	—	—	Low-med	Low	3
NeMo MSDD container	—	—	Medium	Medium	3
NeMo diarization provider	—	—	Low-med	Low	3

Optimization	TTS Speedup	Pipeline Speedup	Effort	Risk	Phase
Parakeet container	~10x ASR	~15%	Medium	Medium	4
Parakeet provider	—	—	Low-med	Low	4
Batch Python scripts	~20% (Edge/Piper)	~8%	Medium	Low	5
Persistent Python pool	~25% (Edge/Piper)	~10%	Med-high	Medium	5
Decompose ViewModel	—	—	Medium	Low	6
Streaming pipeline	—	~30-50%	High	Medium	6
Server-side batching	~40-60% (GPU)	~15%	High	Medium	6
Constructor cleanup	—	—	Low	Low	6
Clean shutdown	—	—	Medium	Low	6

Appendix B: Local vs Cloud Provider Map

Stage	Provider	Compute Profile	Network	Token Required
ASR	faster-whisper	CPU / GPU (container)	localhost only	No
ASR	Parakeet-TDT (new)	GPU (container)	localhost only	No
Translation	NLLB / CTranslate2	CPU / GPU (container)	localhost only	No
TTS	Piper	CPU (subprocess)	None	No
TTS	XTTS v2	GPU (container)	localhost only	No
TTS	Qwen TTS	GPU (container)	localhost only	No
TTS	Edge TTS	Cloud (subprocess)	Microsoft servers	No

Stage	Provider	Compute Profile	Network	Token Required
TTS	OpenAI TTS	Cloud (HTTP)	OpenAI API	API key
TTS	ElevenLabs	Cloud (HTTP)	ElevenLabs API	API key
Diarization	pyannote (legacy)	CPU / GPU	None	HF token
Diarization	NeMo MSDD (new)	GPU (container)	localhost only	No
Diarization	WeSpeaker (new)	CPU (container)	localhost only	No